Corelang

Research Analysis and Preliminary Outline

An Adaptive, Self-Modeling Meta-Language Encompassing Prior Programming Languages

Prepared for Omniversal Architect Labs · October 13, 2026

1. Landscape Summary of Programming Languages

1.1 How Many Programming Languages Exist?

There is no single authoritative census. Estimates depend on definition (general-purpose vs. domain-specific, executable vs. markup, active vs. historical, practical vs. esoteric).

- Documented historical total: The Historical Encyclopedia of Programming Languages (HOPL) catalogs approximately 8,945 languages, including experimental, obsolete, and current entries.
- Wikipedia's list of notable programming languages contains roughly 300 entries (excluding esoteric languages, BASIC dialects, and pure markup languages).
- TIOBE Index tracks more than 150 languages in its popularity dataset; only the top 20–50 receive meaningful commercial or open-source attention.
- Esoteric programming languages (esolangs): The esolang wiki and related collections document well over 2,500 such languages designed for art, experiment, or obfuscation rather than production use.
- Practical active languages: A few dozen languages account for the overwhelming majority of professional software. Stack Overflow and GitHub data consistently show JavaScript, Python, TypeScript, SQL, Java, C#, C/C++, and a handful of others dominating day-to-day use.

In short: thousands of languages have been designed; a few hundred have left lasting traces; fewer than fifty are in widespread contemporary use.

1.2 Version and Iteration Histories

Languages exhibit highly variable versioning practices. Older languages often accumulate multiple formal standards; modern languages frequently follow calendar-driven or semantic-versioning cadences.

Python

Origin: 0.9 (February 1991). Major series: 1.x (1994–2000), 2.x (2000–2020, EOL 2020), 3.x (2008–present). As of October 2026, Python 3.15.0 is the current stable release, with 3.16 in early development. Since Python 3.9 the project has followed an annual October release cadence. Each feature release receives roughly two years of bugfix support followed by three years of security support. The 2-to-3 transition remains the most significant language-level discontinuity in its history.

C

K&R C (1978), ANSI C89 / ISO C90, C99 (1999), C11 (2011), C17 (2018, defect corrections only), C23 (ISO/IEC 9899:2024). C has evolved through a small number of major formal standards rather than frequent feature releases. Backward compatibility remains a dominant design constraint.

Java

JDK 1.0 (January 1996) through Java 27 (September 2026). Since Java 10 the platform has used a six-month feature-release cadence, with long-term support (LTS) releases every two years (current LTS includes Java 21 and Java 25). Language and platform evolution has included generics (5), lambdas

(8), modules (9), records and pattern matching, virtual threads (21), and ongoing work on value types and foreign-function interfaces.

Other notable patterns include ECMAScript edition numbering for JavaScript/TypeScript, Rust's six-week release train, and the long stability of languages such as COBOL and Fortran, which continue to receive incremental standard revisions decades after their original design.

1.3 Rules, Constraints, and What Languages Permit or Forbid

Every language defines a boundary between the legal and the illegal. These boundaries fall into several recurring categories:

- Lexical and syntactic rules: Reserved keywords (C has approximately 32; Python 35 plus soft keywords; COBOL historically hundreds), identifier formation rules, operator precedence, and statement structure.
- Type and semantic constraints: Static vs. dynamic typing, strong vs. weak typing, nominal vs. structural typing, and the presence or absence of null safety, ownership systems, or effect tracking.
- Runtime and operational constraints: Evaluation order, memory model, concurrency primitives, undefined behavior (especially in C/C++), and resource management.
- Expressiveness boundaries: What abstractions are first-class (functions, types, modules, macros) and what must be simulated or deferred to libraries.

Crucially, these rules are not absolute across the historical corpus. Languages differ in which constraints they treat as hard, which they soften over time, and which they deliberately violate for expressive or performance reasons. The existence of esolangs further demonstrates that almost any conventional constraint can be inverted or removed while still yielding a computationally interesting system.

1.4 Evolutionary Trajectories

Observed trends across the corpus include movement toward memory safety without sacrificing performance (Rust), gradual typing and better tooling (TypeScript, Python annotations), improved concurrency models (virtual threads, structured concurrency, ownership), and tighter integration with external systems (HTML/DOM, foreign function interfaces, WebAssembly). Unmaintained and historical languages often preserve designs that later languages abandoned or refined; they therefore remain valuable as sources of structural alternatives.

2. Corelang Preliminary Outline

2.1 Purpose and Scope Statement

Corelang is a meta-language whose working model encompasses every language that has come before it. It does not aim to replace any particular language for ordinary programming. Instead, it maintains an explicit, versioned model of prior languages' syntax, semantics, constraints, error conditions, and evolutionary paths. Every rule, every prohibition, and every error condition inside that model is provisional: subject to modeling, variant generation, viability testing, and selective adaptation. Corelang generates structural variants, tests them, predicts plausible next versions of existing languages, and models adversarial versions of itself for continuous improvement. It can examine code structures from historical and contemporary languages, from HTML and other markup, and from language-model outputs, and it can outline those structures inside its own model.

2.2 Core Principles

1. Non-static rules. Syntax, reserved words, type constraints, and error conditions are data. They can be examined, varied, and redefined under controlled conditions while the change itself is recorded.
2. Self-modeling. Corelang maintains an internal representation of its own current rule set, capabilities, and limitations, as well as models of prior languages.
3. Structure extraction. It analyzes and outlines code structures generated in earlier languages and in web/markup structures, mapping them into its own model.
4. Variant generation and viability testing. It produces syntactic, semantic, and structural variants and evaluates them for consistency, executability under a modeled runtime, and coherence with stated goals.
5. Predictive evolution. It asks what a language was designed to do and what languages appear to be evolving toward, then builds working models of plausible next versions by running self-generated algorithms across core and unmaintained languages.
6. Adaptive constraint handling. A construct that “cannot run” under the current rule set can be made to run by an explicit, audited rule adaptation. Conversely, new restrictions can be introduced and modeled.
7. Adversarial self-modeling. Corelang generates rule-set variants that stress or contradict its current model and uses the results to improve robustness and coverage.

2.3 Conceptual Architecture

At a high level, Corelang consists of the following interacting components:

- Language Model Store: Versioned representations of prior languages (grammars, keyword sets, type rules, error models, standard library surfaces, evolutionary history).
- Self-Model: An explicit, queryable representation of Corelang’s own current rules, open constraints, and adaptation history.
- Structure Extractor: Processes source text, HTML, or other artifacts and produces abstract structural outlines (AST-like or higher-level pattern descriptions).
- Variant Generator: Produces candidate rule or program modifications according to selected strategies (relaxation, restriction, recombination of features from other languages, adversarial mutation).
- Viability Tester: Evaluates variants for internal consistency, modeled executability, and alignment with declared goals.
- Evolution Predictor: Runs comparative algorithms across the Language Model Store (including unmaintained languages) to surface recurring patterns, abandoned designs, and plausible future structures.
- Adaptation Ledger: Records every provisional rule change, its justification, the variants that were tested, and the resulting model state. Changes are branchable and reversible.

2.4 Key Operational Processes

Ingest and Model

Given a language description, grammar, or corpus, construct or update a model entry. Capture reserved words, syntactic forms, type rules, and known error conditions as first-class, queryable data.

Extract and Outline Structures

Process source from prior languages or HTML and emit human-readable or machine-readable outlines of the structures present. These outlines become part of the model and can later serve as seeds for variant generation.

Generate and Test Variants

Select a constraint (for example, “keyword X is reserved”) and produce one or more variants in which that constraint is relaxed, strengthened, or redefined. Score each variant for viability. Retain or discard according to policy; always record the experiment.

Adapt Rules

When a viable adaptation is accepted, update the Self-Model and write an entry to the Adaptation Ledger. The previous state remains available for comparison or rollback.

Predict Next Versions

Run comparative or evolutionary algorithms across the store of core and unmaintained languages. Output a working model of plausible next-version forms for a target language, together with the patterns that support the prediction.

2.5 Example Scenarios

Scenario A: Variable as Container vs. Uncontainable Object

Observation: Most languages treat variables as containers that hold values or references. Requirement: Model an entity that cannot be placed inside an ordinary variable. Corelang response: Introduce (or model the introduction of) a first-class “uncontainable” or “non-variable” construct, record the rule change, generate test cases that attempt to place the entity in a variable, and document the resulting semantics. The original container model remains available for comparison.

Scenario B: Restricted Keyword Modeling

Observation: “if” (or any other term) is a reserved keyword in language L. Corelang response: Treat the reservation as a provisional rule. Generate a variant in which the term is available as an identifier, evaluate parsing and semantic consequences, and record whether the variant is viable for a given purpose. The restriction can be restored or left in a branched state.

Scenario C: “This Cannot Run” → “Now It Can”

A construct is rejected under the current rule set. Corelang identifies the specific constraint that causes rejection, generates a minimal adaptation that removes or redefines that constraint, tests the adapted model for consistency, and, if viable, records the change. The original non-runnable state remains documented.

Scenario D: Adversarial Improvement

Corelang generates a rule-set variant that deliberately contradicts or stresses its current Self-Model (for example, by removing all keyword reservations or by inverting evaluation order). The adversarial model is used to discover blind spots, after which improvements are folded back into the primary model under ledger control.

2.6 Risks, Safeguards, and Traceability

Because rules are provisional by design, Corelang must treat consistency, explainability, and reversibility as first-class concerns. Every adaptation is ledgered. Models are versioned. Divergent branches can be retained or discarded. The system prefers audited, minimal changes over unconstrained mutation. These safeguards keep the working model coherent while still permitting the adaptive behavior that is the point of the design.

2.7 Open Questions and Next Research Steps

8. What is the minimal set of primitives needed to represent arbitrary prior language rules as data while remaining computationally tractable?

9. How should viability be scored in the absence of a concrete runtime—purely by consistency, by simulated execution, or by both?
10. Which corpora of unmaintained languages and HTML structures yield the highest value for predictive modeling?
11. What governance model should govern acceptance of an adaptation into the primary Self-Model versus retention as a branch?
12. How can Corelang's outputs (outlines, predicted next versions, adversarial analyses) be made most useful to human designers working on ordinary languages?

3. Glossary of Corelang Terms

Adaptation Ledger — The append-only record of every provisional rule change, its justification, tested variants, and resulting model state.

Self-Model — Corelang's explicit, queryable representation of its own current rules and limitations.

Uncontainable — A construct deliberately designed so that it cannot be stored in an ordinary variable or container under the current model.

Viability — The degree to which a generated variant remains internally consistent, executable under a modeled runtime, and coherent with stated goals.

Variant — A candidate modification of a rule, syntax form, or program structure produced for testing.

4. Closing

The research shows that programming languages are numerous, historically layered, and governed by rules that have always been subject to revision across versions and across languages. Corelang takes that historical fact as a design premise: rules are data, models are versioned, and adaptation is an explicit, auditable process rather than an external evolution that happens to the language. The outline above supplies a preliminary architecture and set of operational behaviors sufficient to begin further refinement, implementation experiments, or collaborative design work.

— End of Preliminary Outline —